

Техническое задание (ТЗ): Разработка SolanaPay-KZ

1. Название проекта

****SolanaPay-KZ**** [1].

2. Цель проекта

Разработать open-source SDK и готовые плагины для CMS Tilda и WooCommerce, которые позволят малому и среднему бизнесу в Казахстане принимать платежи в криптовалюте (USDC/SOL) с автоматической конвертацией из тенге (KZT) по актуальному курсу [1].

3. Описание проблемы

Для предпринимателей и фрилансеров в Казахстане прием международных платежей связан с высокими комиссиями, задержками и риском блокировок. На популярных платформах, таких как Tilda и WordPress (WooCommerce), отсутствует простое, готовое решение для приема платежей в стейблкоинах (например, USDC) на блокчейне Solana без посредников и сложных интеграций [1].

4. Предлагаемое решение

Создать комплексное решение, состоящее из двух ключевых частей [1]:

1. ****Core SDK:**** Библиотека, которая инкапсулирует логику конвертации KZT в USDC/SOL и генерирует QR-код для оплаты через Solana Pay.
2. ****Готовые интеграции:**** Плагины и скрипты для легкого подключения к сайтам на WooCommerce и Tilda.

Процесс для конечных пользователей будет выглядеть так: продавец получает USDC напрямую на свой кошелек (например, Phantom) с минимальной комиссией, а покупатель оплачивает товар, просто отсканировав QR-код своим мобильным кошельком [1].

5. Ключевые компоненты и архитектура

5.1. Core SDK (Основная библиотека на JS/TS)

Это будет ядро всей системы, реализованное в виде JS/TS модуля.

* ****Функционал:****

* Принимает на вход сумму в KZT.

* Запрашивает актуальный курс KZT/USDC через API надежной биржи (например, Binance, Bybit или локальные казахстанские обменники). Необходимо определить и интегрировать наиболее

стабильный источник. Реализовать механизм **fallback (резервного варианта)**. Например: "Система должна сначала попытаться получить курс через API CoinGecko. Если запрос неудачен, она должна автоматически переключиться на резервный API, например, Binance".

- * Конвертирует сумму из KZT в USDC.
- * Генерирует стандартный QR-код в соответствии со спецификацией Solana Pay, включающий адрес получателя, сумму и другие необходимые параметры.
- * ****Требования:****
 - * Код должен быть опубликован в открытом репозитории на GitHub [1].
 - * Библиотека должна быть хорошо документирована для дальнейшего использования сторонними разработчиками.

5.2. Плагин для WooCommerce

- * ****Функционал:****
 - * Интеграция в качестве нового платежного шлюза в WooCommerce.
 - * В административной панели WordPress должна быть страница настроек плагина, где продавец может указать свой Solana-адрес для приема платежей [1].
 - * На странице оформления заказа (checkout) при выборе этого способа оплаты должен вызываться Core SDK для генерации и отображения QR-кода с нужной суммой.
 - * Реализовать механизм проверки статуса транзакции на блокчейне Solana для подтверждения успешной оплаты.
 - * Указать, какой уровень подтверждения (`commitment level`) транзакции в сети Solana считать достаточным для подтверждения заказа. Чаще всего для платежей используют уровень `finalized`, чтобы избежать риска отката транзакции.

5.3. Интеграция с Tilda

- * ****Функционал:****
 - * Создание виджета или скрипта, который можно встроить на сайт Tilda [1].
 - * Интеграция должна работать через кастомные платежные шлюзы Tilda или механизм Webhooks [1].
 - * Подготовка детальной и понятной инструкции для пользователей Tilda (не-разработчиков) по установке и настройке скрипта.

5.4. Требования к безопасности

Плагины и SDK ни при каких обстоятельствах не должны запрашивать, хранить или обрабатывать приватные ключи пользователей. Вся работа с кошельками должна происходить на стороне клиента (в браузере) в соответствии со стандартами Solana Wallet Adapter".

6. Технологический стек

- * **Язык программирования:** TypeScript.
- * **Основной SDK для Solana:** Рекомендуется использовать современный `@solana/kit` и его плагины (`@solana/kit-plugin-rpc`, `@solana/kit-plugin-wallet` и др.) [2].
- * **Среда выполнения:** Node.js [3].
- * **Управление зависимостями:** Yarn или npm [3].
- * **Для плагина WooCommerce:** PHP, WordPress Plugin API.

"Настройка среды разработки"

Чтобы у разработчика не возникло вопросов, с чего начать, можно добавить краткую инструкцию по настройке окружения.

- **Рекомендация:** Добавить пункт, описывающий установку всех необходимых инструментов (Rust, Solana CLI, Node.js) с помощью единой команды, как указано в документации [3]:

```
curl --proto '=https' --tlsv1.2 -sSfL https://solana-install.solana.workers.dev | bash
```

7. План реализации (Milestones)

1. **Разработка Core SDK:** Написание логики конвертации и генерации QR-кодов. Создание публичного репозитория на GitHub [1].
2. **Создание плагина для WooCommerce:** Разработка, интеграция с админ-панелью и процессом оплаты [1].
3. **Интеграция с Tilda:** Создание скрипта и написание инструкции по его подключению [1].
4. **Тестирование и документация:** Запуск демо-магазина, написание гайдов на русском и казахском языках и официальный релиз проекта [1].

8. Ожидаемые результаты

1. Публичный GitHub-репозиторий с Core SDK.
2. Рабочий плагин для WooCommerce, доступный для установки.
3. Скрипт и подробная инструкция для интеграции с Tilda.
4. Демонстрационный сайт, показывающий работу решения.
5. Пользовательская документация на русском и казахском языках.

9. Дополнительные рекомендации

Вы предоставили несколько ссылок на готовые шаблоны с `solana.com/developers/templates`. Рекомендую изучить их перед началом разработки. Особенно полезными могут оказаться:

- * ``eve-pay``: для общей логики обработки платежей. <https://solana.com/developers/templates/eve-pay>
- * ``kit-node-solanax402``: как пример бэкэнд-сервиса с использованием `@solana/kit`. <https://solana.com/developers/templates/kit-node-solanax402>
- * ``x402-template`` (Next.js): может стать отличной основой для демо-магазина. <https://solana.com/developers/templates/x402-template>

Анализ этих проектов может значительно ускорить разработку, так как часть необходимой логики (например, создание и отправка транзакций) уже может быть реализована.

Исходя из своего опыта, для получения курса KZT/USDC я бы порекомендовал начать с API одной из крупных международных бирж, так как они обычно наиболее стабильны и хорошо документированы.

Вот несколько вариантов, отсортированных по надежности и простоте интеграции:

1. **Binance API:**

- **Надежность:** Очень высокая. Это крупнейшая биржа с огромной ликвидностью, что обеспечивает рыночный и стабильный курс.
- **Простота:** У них есть публичный REST API, который не требует ключей для получения рыночных данных (тикер цены). Найти пару KZT/USDT или KZT/USDC может быть сложно, поэтому чаще всего используют "синтетический" курс через USDT: сначала получают курс KZT/USDT, а затем USDT/USDC (который почти всегда 1:1). Это стандартная практика.

2. **CoinGecko API:**

- **Надежность:** Высокая. Это агрегатор данных со множества бирж.
- **Простота:** Очень простой в использовании. У них есть бесплатный и щедрый тарифный план, который идеально подходит для такого проекта. Скорее всего, у них будет прямой тикер для нужной пары.

Моя рекомендация:

Начните с **API CoinGecko**. Он, как правило, самый простой для быстрого старта и не требует сложных аутентификаций для получения цены. Если по какой-то причине его будет недостаточно, **API Binance** — это промышленный стандарт и самый надежный вариант.